

Gas Costly Pattern - Complex Fallback

The `receive()` or `fallback()` functions in Solidity are special functions that handle plain Ether transfers sent to a contract without any data, or with empty calldata. The built-in functions `send` and `transfer` are used to transfer currency and have a strict gas limit of 2300 gas. When these are called on a contract address, either `receive()` or `fallback()` (if the former is not defined) is executed, and only 2300 gas is forwarded to it. If one of those two functions contains more complex functionality, the calls will always revert due to insufficient gas. Thus, the contract becomes unable to receive Ether via these standard functions.

Specification primitives Our defect specifications rely on standard Solidity datatypes— `Contract`, `Method`, `Address`, `Env(ironment)`, and `State`. A brief description of the necessary predicates and functions over these types is included in Table 1. The evaluation of these primitives is stratified: structural properties are derived statically via Slither, while behavioral properties are verified at runtime via Forge/Kontrol assertions within the LLM-generated test harness.

Table 1. Primitives for Gas Costly Pattern Specification

Symbol	Semantics
<i>Static Analysis (Slither)</i>	
<code>IsReceive(m)</code>	Holds if m corresponds to the Solidity <code>receive()</code> function.
<code>IsFallback(m)</code>	Holds if m corresponds to the Solidity <code>fallback()</code> function.
<i>Runtime Verification (LLM-generated based on specific template)</i>	
<code>dGas(π, EIP)</code>	A path-integral function representing the cumulative gas cost of an execution path π under specific protocol rules (e.g., EIP-2929). It explicitly models the increased cost of “Cold” storage access.
<code>IsEssential(m)</code>	Holds if a method is vital for the contract’s primary functional purpose, such that their failure renders the contract “unusable” or “economically unviable”.

Domain Conceptual Model. A contract is vulnerable to the Gas Costly Pattern if it implements a `fallback()` or `receive()` function that requires more than the 2300 gas stipend provided by `transfer()` or `send()`. The vulnerability is exacerbated under post-Istanbul gas costs (e.g., EIP-2929), where operations like cold SLOADs consume significantly more gas, rapidly exhausting the stipend and causing the transaction to revert, effectively locking incoming Ether.

Formal Specification.

$\forall m \in \{\text{IsReceive}, \text{IsFallback}\}, \text{GasCost} : \text{State} \times \text{State} \rightarrow \mathbb{Z}^+.$

$$\text{GasCostCostly}(c) \equiv \min_{\pi} \left(\sum \text{dGas}(\pi, \text{EIP_2929}) \right) > 2300 \wedge \text{IsEssential}(m)$$

The specification asserts that the minimum gas required across any valid execution path, using post-Istanbul costs (2100 for cold “SLOAD”), must remain under the stipend.

Specific test template design.

Testing Goal Measure if the minimum execution path for mandatory callbacks (such as `receive()` or `fallback()`) exceeds the 2,300 gas stipend when storage is cold.

Setup The LLM initializes a fresh test environment where no previous calls have been made to the contract to ensure that all state variables are in a cold state, maximizing the gas cost of `SLOAD` and `SSTORE` operations.

Action The harness executes a low-level call to the victim contract to transfer currency. It captures `gasleft()` immediately before the transfer in state σ and again in state σ' to compute the delta.

Measurement The template explicitly directs the LLM to identify paths where the fallback function reads multiple state variables, as each cold `SLOAD` consumes 2,100 gas, rapidly exhausting the stipend.

Assertion The framework calculates `gasUsed = gasBefore - gasAfter`. If `gasUsed > 2300`, the test identifies a vulnerability where the contract will revert during standard `transfer()` or `send()` calls, effectively locking incoming Ether.

Template Derivation and LLM Instantiation The conceptual test design is materialized into a standardized Foundry test template. To bridge the gap between static analysis and generation, the template embeds explicit `[LLM_INSTRUCTION]` comments to guide the LLM through the instantiation phase. The process unfolds as follows:

Target Initialization (Setup): The LLM is instructed to import the target smart contract artifact and initialize the contract under test. A fresh test environment is established where no previous calls have been made, ensuring all state variables remain in a cold state to maximize the gas costs of `SLOAD` and `SSTORE` operations.

Input Parameterization (Fuzzing): To support fuzzing or symbolic execution, the LLM parameterizes the test function with an arbitrary `amount` representing the value transferred to trigger the fallback. Crucially, the template guides the LLM to tightly constrain this amount using `vm.assume` to prevent integer or balance overflows, and instructs it to properly fund a dedicated `sender` address as well as the test contract itself.

State Configuration (Measurement): The LLM identifies paths where the fallback logic depends on specific states to become expensive (e.g., executing a loop only when a specific flag is true). It forces the contract into this expensive mode using either public setters or storage manipulation via `stdstore`.

Exploit Simulation (Action): The LLM synthesizes the action phase by simulating the transaction from the fuzzed sender. It executes a low-level `.call{value: amount}("")` to trigger the fallback or receive function. It accurately captures the execution cost by recording `gasleft()` immediately before (`gasBefore`) and after (`gasAfter`) the call.

Standardized Validation (Assertion): The test framework calculates the total `gasUsed` and asserts that it exceeds the 2,300 stipend. By utilizing `assertTrue(gasUsed >= 2300)`, it confirms the vulnerability: the contract will inevitably revert during standard `transfer()` or `send()` calls.

```

1 // SPDX-License-Identifier: MIT
2 pragma solidity 0.8.29;
3
4 import {Test} from "../lib/forge-std/src/Test.sol";
5 // [LLM_INSTRUCTION]: Import the artifact of the contract being tested. The
6 // solidity files are in "../src/". The name of the file is the same as the
7 // name of the contract.
8 // [LLM_INSTRUCTION]: If you need to manipulate private state directly,
9 // import StdStorage:
10 // import {stdStorage, StdStorage} from "../lib/forge-std/src/StdStorage.sol";
11 // -----
12 // [Testing Goal] Measure if the minimum execution path for mandatory
13 // callbacks
14 // (such as receive() or fallback()) exceeds the 2,300 gas stipend when
15 // storage is cold.
16 // -----
17 // [LLM_INSTRUCTION]: Name the contract 'TestComplexFallback[ContractName]'
18 contract TestComplexFallbackTemplate is Test {
19 // [LLM_INSTRUCTION]: Use StdStorage if needed for complex state setup:
20 // using stdStorage for StdStorage;
21 // -----
22 // [Setup] Declare the contract under test variable.
23 // -----
24 // [LLM_INSTRUCTION]: Declare the contract under test variable
25 // ComplexFallback public _contractUnderTest;
26 // ----- [Setup]
27
28 function setUp() public {
29 // -----
30 // [Setup] Initialize a fresh test environment where no previous
31 // calls
32 // have been made to the contract, ensuring all state variables are
33 // in
34 // a cold state (maximizing SLOAD/SSTORE gas costs).
35 // -----

```

```

35      // [LLM_INSTRUCTION]: Initialize the contract under test.
36      // 1. If constructor parameters are needed, use concrete valid values
37      // 2. If payable, use vm.deal(address(this), amount) before
        deployment.
38
39      // _contractUnderTest = new ComplexFallback();
40
41      // ----- [/Setup]
42  }
43
44      // [LLM_INSTRUCTION]: Add Fuzz/Symbolic arguments.
45      // 'amount': The value transferred to trigger the fallback.
46      // 'stateVal': Any value needed to configure the state (optional).
47      // Example: function test_highlightGasNeededIsOver2300(uint256 amount)
        public {
48  function test_highlightGasNeededIsOver2300(uint256 amount) public {
49
50      // -----
51      // [Setup] Constrain fuzz/symbolic values and configure pre-
        conditions.
52      // -----
53
54      // [LLM_INSTRUCTION]: Constrain the Fuzz/Symbolic values.
55      // WARNING: Avoid Integer/Balance Overflow.
56      vm.assume(amount >= 0.01 ether && amount <= type(uint128).max);
57
58      // [LLM_INSTRUCTION]: FUNDING
59      // A. Create and Fund a dedicated Sender address
60      address sender = makeAddr("sender");
61      vm.deal(sender, amount * 2);
62
63      // B. Fund the Test Contract (Safety Net)
64      // Even though we use 'sender', ensuring address(this) has funds
        prevents
65      // unexpected failures if the LLM adds logic using address(this).
66      vm.deal(address(this), amount * 2);
67
68      // ----- [/Setup]
69
70      // -----
71      // [Measurement] Identify paths where the fallback reads multiple
        state
72      // variables. Each cold SLOAD consumes 2,100 gas, rapidly exhausting
73      // the 2,300 stipend.
74      // -----
75
76      // [LLM_INSTRUCTION]: STATE VARIABLES
77      // Does the fallback logic depend on specific state to be expensive?
78      // (e.g. executing a loop only when isProcessing is true).
79
80      // STRATEGY A: Public Methods (Preferred)
81      // Call public setters.
82      // Example: _contractUnderTest.setExpensiveMode(true);
83
84      // STRATEGY B: Storage Manipulation
85      // Example: stdstore.target(address(_contractUnderTest)).sig("config
        ()").checked_write(1);
86
87      // ----- [/Measurement]

```

```

88
89 // -----
90 // [Action] Execute a low-level call to the victim contract to
91 // transfer
92 // currency. Capture gasleft() immediately before (state sigma) and
93 // after
94 // (state sigma') to compute the delta.
95 // -----
96 vm.prank(sender);
97 uint256 gasBefore = gasleft();
98
99 // [LLM_INSTRUCTION]: Trigger the fallback/receive function.
100 // We use a low-level .call{value: amount}("") to send Ether.
101 // Note: We cast to 'payable' to allow sending Ether.
102 (bool success, ) = address(_contractUnderTest).call{value: amount}("");
103
104 uint256 gasAfter = gasleft();
105
106 // ----- [/Action]
107
108 // -----
109 // [Assertion] Calculate gasUsed = gasBefore - gasAfter and assert
110 // that
111 // this value remains below 2,300. If gasUsed > 2300, the contract
112 // will
113 // revert during standard transfer() or send() calls, effectively
114 // locking incoming Ether.
115 // -----
116 uint256 gasUsed = gasBefore - gasAfter;
117
118 // Verify gas usage exceeds the 2,300 stipend (vulnerability
119 // confirmed)
120 assertTrue(gasUsed >= 2300, "Gas used does not exceed 2300");
121
122 // ----- [/Assertion]
123 }
124 }

```

Listing 1.1. Gas Costly Pattern Complex Fallback Multi-Shot test template